NeuroSnap: Vision-Guided Learned Trajectory Generation for Autonomous Quadrotor Gate Navigation

Project Implementation Report

Name: Aryan Amit Tiwari

Registration No.: 23BAI1409

Course: BCSE428L – Autonomous Drones

1. Project Links

Source code (GitHub)	https://github.com/AryanAT13/Neurosnap
Repository README	README.md (results, figures, how to run)
Demo video	docs/media/demo.mp4 and docs/media/demo.gif
Raw results (CSV / JSON)	results/ (benchmark.csv, benchmark_summary.json, planner_eval.json, perception_eval.json, dagger.json, control_analysis.json)
Trained models	models/ (planner_dagger.pt, gatenet.pt, residual.npz and training logs)
Initial report (proposal)	Drone_Project_Report_Aryan_Amit_Tiwari_23BAI1409.pdf
Reference paper (IEEE)	https://doi.org/10.1109/ICRA.2011.5980409

2. Abstract

This report describes the implementation of **NeuroSnap** (*Neural minimum-Snap*), the project proposed in the initial report. A simulated Crazyflie-class quadrotor must fly through a randomised course of gates using only a forward-facing camera and an approximate, noisy map of the track. A small convolutional network (GateNet) finds the four corners of every gate in each frame. PnP with the known gate size turns the corners into 3-D gate positions, and a Kalman filter fuses them with the prior map. A Transformer trajectory generator then replaces the reference paper's minimum-snap optimisation: in a single forward pass it predicts segment durations and knot derivatives, which a fixed Hermite decoder turns into a smooth, C3-continuous trajectory. A cascaded PID controller with a learned drag feed-forward flies the trajectory on a full 6-DOF model.

Across 440 simulated missions on 40 unseen courses, the full stack completed **97.5 %** of courses, against 100 % for the classical minimum-snap planner with the same perception and 0 % when flying the prior map alone. It tracked its reference with 1.7 cm RMSE (classical: 4.0 cm). Offline, the learned time allocation is within 0.1 % of the optimiser, and in hybrid mode the trajectories match it (1.00× optimal snap) at 2.8× lower compute. Two additions made the learned planner work in closed loop: **DAgger** against receding-horizon drift, and an analytic **plan verifier**. The detector reaches 0.89 precision and 0.89 recall, and one camera frame is processed in 8.7 ms on a laptop CPU.

3. Reference Paper and Objective (recap)

The project builds on D. Mellinger and V. Kumar, "*Minimum Snap Trajectory Generation and Control for Quadrotors*", IEEE ICRA 2011, pp. 2520–2525 (Best Conference Paper). The paper uses differential flatness to plan in the space of position and yaw. It represents the trajectory as piecewise polynomials, minimises the integral of squared snap as a quadratic program, optimises the segment durations, and tracks the result with a nonlinear controller. The method is powerful but mathematically heavy, assumes the waypoints are given, and must be re-solved whenever they change.

The objective set in the initial report was a software pipeline with these parts:

1. detect gates from a single onboard camera with a deep network and estimate their pose;
2. convert detections into 3-D waypoints and fuse them with a prior map;
3. generate a smooth, dynamically feasible trajectory with a neural network trained by imitation of the minimum-snap solver, replacing the run-time QP and time allocation;
4. track it with a PID controller, optionally with a learned residual model, behind a safety layer;
5. evaluate detection quality, trajectory quality, tracking error, success rate and compute time against the classical minimum-snap planner and a straight-line planner.

All five parts are implemented and evaluated below. The learned residual model, listed as a stretch goal, is also complete.

> **Deviations from the proposal.** (1) The proposal named gym-pybullet-drones. The project instead uses its own 6-DOF simulator with the same Crazyflie CF2X parameters, because a self-written renderer gives exact corner labels for every frame and the PyBullet build was not needed for anything else. (2) The proposal named a YOLO-family detector with a pose-regression head. GateNet is a CenterNet-style single-stage detector that regresses the four opening corners directly; PnP then gives the exact pose from geometry instead of a learned regressor. (3) DAgger and the plan verifier were not in the proposal; they were added after the first closed-loop results (Section 10).

4. System Overview

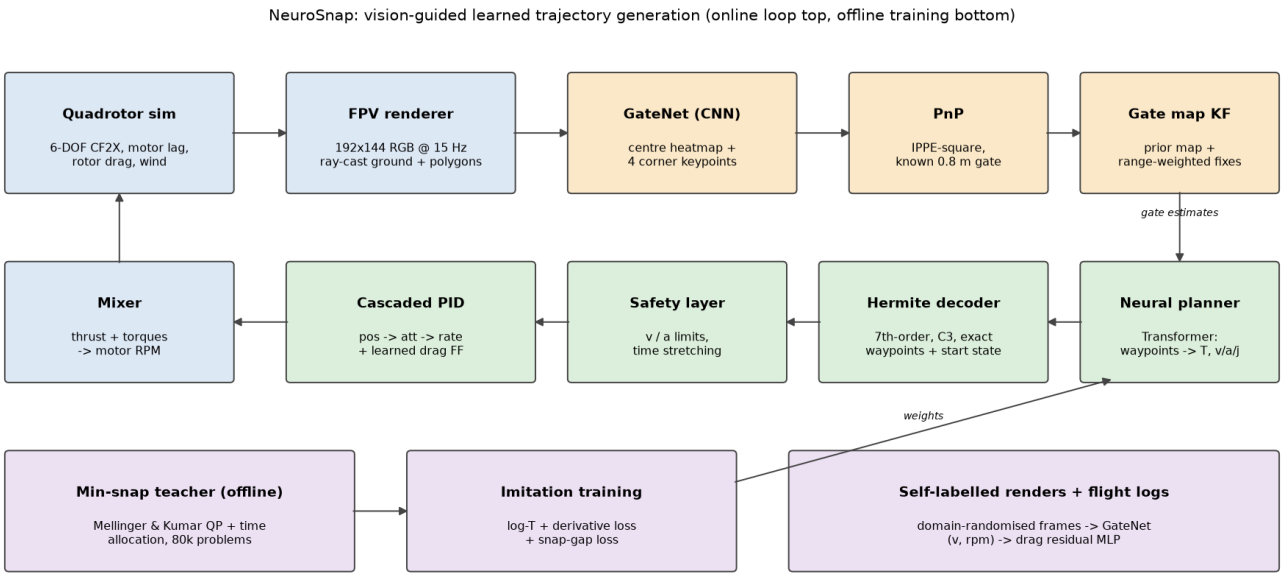


Figure 1. NeuroSnap architecture. Top two rows: the online loop (camera and perception at 15 Hz, re-planning at 10 Hz, control at 500 Hz, physics at 1 kHz). Bottom row: offline training.

The system is written in Python (NumPy, SciPy, PyTorch, OpenCV, Matplotlib) and organised as independent modules:

- a simulated quadrotor and FPV camera producing images;
- a perception module producing gate detections and PnP poses;
- a gate map (Kalman filter) producing gate-position estimates;
- a planning module producing time-parameterised reference trajectories, checked by the verifier;
- a cascaded flight controller producing motor commands.

Every component was built from scratch for this project: the simulator and renderer, the minimum-snap teacher, the planner dataset, both neural networks and the residual model, DAgger, the mission runner, the benchmark and the dashboard. The code is about 3,500 lines with 14 unit and integration tests.

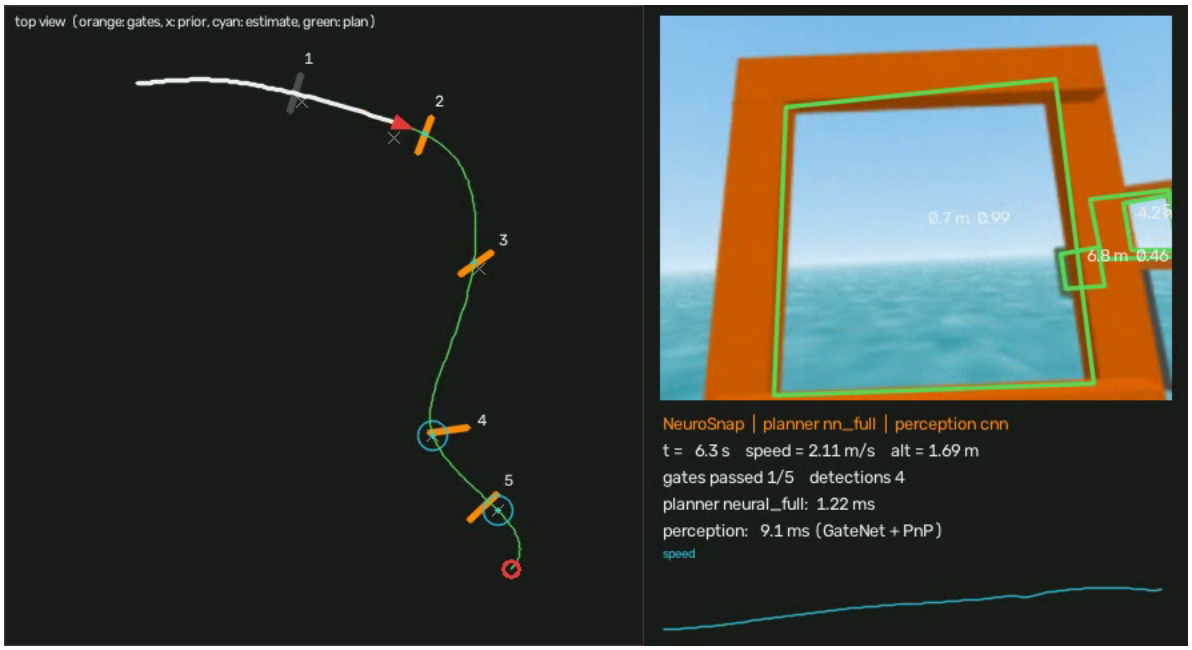


Figure 2. The live dashboard 6.3 s into a mission. Left: top view with the true gates (orange), the prior map (x), the current gate estimates with 1-sigma circles (cyan), the current plan (green) and the flown path (white). Right: the onboard camera with GateNet detections and PnP ranges, and the flight HUD.

5. Implementation

5.1 Quadrotor model (Syllabus Modules 1 and 4)

The vehicle is a rigid-body Newton–Euler model of the Bitcraze Crazyflie 2.x in X configuration, with the parameters of the gym-pybullet-drones CF2X model. Each rotor produces thrust $T_i = k_f\, n_i^2$ and drag torque $Q_i = k_m\, n_i^2$ (n in RPM). Rotor speed follows the command through a first-order lag, and rotor drag acts against the body-frame velocity in proportion to the summed rotor speed:

$$m\, dv/dt = R\,[0,\ 0,\ \Sigma T_i]^T - m\, g\, e_z + F_{drag} + F_{ext}, \quad J\, d\omega/dt = \tau - \omega \times J\omega, \quad dn_i/dt = (n_{i,cmd} - n_i)\,/\,\tau_m$$

The state is integrated with semi-implicit Euler at 1 kHz, with quaternion attitude. Each mission draws a random constant wind (σ = 0.3 m/s), and the controller sees position and velocity with small Gaussian noise.

Parameter	Value	Parameter	Value
Mass	27 g	Airframe / arm length	Quad-X / 39.7 mm
Inertia (Ixx, Iyy, Izz)	1.4, 1.4, 2.17 × 10^{-5} kg·m²	k_f / k_m	3.16×10^{-10} / 7.94×10^{-12}
Thrust-to-weight	2.25	Motor time constant	20 ms
Hover / max rotor speed	14,476 / 21,714 RPM	Rotor drag (xy, z)	9.18 / 10.31 ×10^{-7}

Parameter	Value	Parameter	Value
Camera	192×144 px, 90° HFOV, tilted 15° up, 15 Hz	Physics / control rate	1 kHz / 500 Hz
Gates	0.8 m opening, 0.15 m frame, on a stand	Limits (planner)	v ≤ 3 m/s, a ≤ 6 m/s²

5.2 Flight controller (Syllabus Modules 2 and 3)

The reference paper tracks its trajectories with a hand-designed nonlinear controller. NeuroSnap uses a standard PX4/DSL-style cascade instead:

- position and velocity PID with acceleration feed-forward from the trajectory, giving a desired thrust vector (tilt limited to 45°);
- desired attitude from the thrust direction and the yaw reference (heading towards the next gate);
- attitude P on the SO(3) error $e_R = \tfrac{1}{2}\,\mathrm{vee}(R_d^T R - R^T R_d)$, giving body-rate set-points (≤ 12 rad/s);
- body-rate PI and a mixer that inverts the 4×4 thrust/torque allocation matrix.

The position integrator uses conditional integration: only errors under 25 cm are integrated. An early version wound up on large steps and settled slowly (Section 10). The residual model of Section 5.9 is subtracted from the commanded acceleration as a feed-forward term.

5.3 FPV camera and self-labelled training data (Syllabus Module 7)

The onboard camera is a small renderer written in NumPy and OpenCV. The ground and sky are ray-cast per pixel onto the plane z = 0 with a procedural texture (noise, checkerboard or tiles), distance fog and a sky gradient. Gates, stands and distractor boxes and pillars are drawn as flat-shaded polygons, with back-face culling, near-plane clipping (Sutherland–Hodgman) and painter's-algorithm ordering. Every scene is domain-randomised: gate colour, sky, ground texture, light direction, fog, brightness, contrast, colour gain, sensor noise and motion blur.

Alongside the image, the renderer outputs exact labels: the projected opening corners (ordered as seen, so a gate seen from behind keeps a consistent corner order) and the gate pose in the camera frame. No manual annotation was needed. Training frames are generated on the fly in four DataLoader workers from camera poses sampled as they occur in flight (between consecutive gates, looking roughly at the next one, with manoeuvring roll and pitch), so GateNet never sees the same image twice.

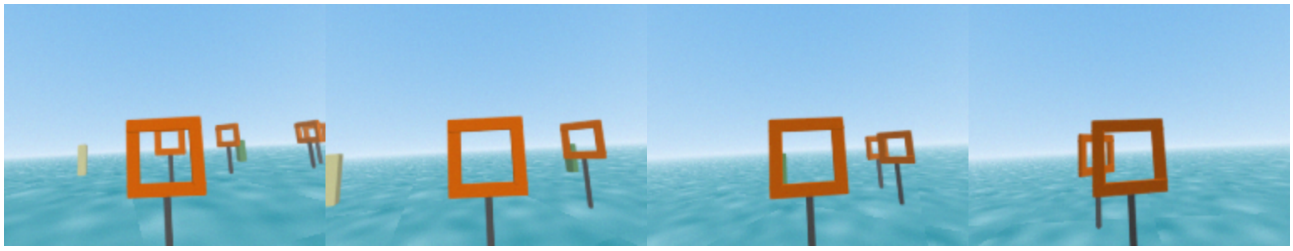


Figure 3. Onboard views rendered with the demo style. The far gates are seen through the openings of the near ones.

5.4 Gate perception: GateNet and PnP

Network	Design	Parameters	CPU time
GateNet	CenterNet-style, anchor-free: residual encoder (5 stages, strides 2–32) with a feature-pyramid decoder to stride 4. Heads: centre heat-map (focal loss), sub-cell offset and 8 corner offsets (L1). Trained on the laptop GPU (Apple MPS), 25 epochs × 250 steps × 32 images = 200k fresh renders, about 35 min.	2.92 M	8.7 ms*

* Including heat-map decoding and PnP for all detections. Peaks are extracted with 3×3 max-pool non-maximum suppression. For each detection the four corners and the known 0.8 m opening go into `cv2.solvePnP` with the IPPE-square solver, which gives the gate centre and normal in the camera frame. Keeping the geometry in PnP means the network only has to learn image keypoints, which transfers far better than regressing 3-D pose directly.

5.5 Gate map (Kalman filter)

Before take-off the drone receives a prior map: every true gate centre plus Gaussian noise ($\sigma = 0.45$ m horizontally, 0.2 m vertically). This is the only information the "blind" ablation has. Each PnP fix within 7 m is transformed into the world frame with the estimated camera pose. It is associated with the nearest not-yet-passed gate by Mahalanobis distance (gate $\chi^2 = 16$) and fused by a per-gate Kalman update. The measurement covariance grows with range, $\sigma(d) = 0.04 + 0.012\, d^2$ m, which follows the PnP depth error measured in Section 7.3.

5.6 Minimum-snap teacher (the reference paper, used offline)

The paper's method is implemented in its unconstrained knot-derivative form. Each segment is a 7th-order polynomial fixed by position, velocity, acceleration and jerk at its two ends. On normalised time $\tau = t/T$ the boundary values become $y_d = x_d\, T^d$, the coefficients are $a = A\, y$ with a constant 8×8 matrix A, and the integrated squared snap is a closed-form quadratic:

$$J = \int (d^4p / dt^4)^2\, dt = T^{-7}\, y^T H\, y, \quad H = A^T Q_1 A, \quad G_{ij}(T) = H_{ij}\, T^{d_i + d_j - 7}$$

With the waypoints and the current velocity and acceleration fixed, the free derivatives (initial jerk and v, a, j at every interior waypoint) follow from one linear solve shared by x, y and z. The segment durations are then optimised on log T with L-BFGS for $F(T) = J^*(T) + \rho\, \Sigma T$ ($\rho = 40$). The gradient comes for free from the envelope theorem: $dJ^*/dT_s = x_s^T\, (dG_s/dT)\, x_s$. A feasibility step stretches the durations and re-solves until $|v| \le 3$ m/s and $|a| \le 6$ m/s². The teacher labelled 80,000 training and 4,000 test problems in 157 s on six cores. Half of the problems start mid-flight from a teacher trajectory with perturbed velocity and acceleration, so re-planning states are covered.

5.7 Neural trajectory generator

The waypoints are expressed in a canonical frame: origin at the current position, x-axis towards the first waypoint. Each waypoint becomes a 16-dimensional token (position, offset to the previous waypoint, segment length, start and end flags, the initial velocity and acceleration on the first token, and its index). A 4-layer pre-norm Transformer encoder ($d = 128$, 4 heads, 568k parameters) maps the tokens to ten outputs per waypoint: log T of the incoming segment and the 3 × (v, a, j) knot derivatives.

The outputs pass through a fixed, differentiable **Hermite decoder**: the same 7th-order parametrisation the teacher uses. The decoder overwrites the positions with the waypoints, the first velocity and acceleration with the current state, and the final derivatives with zero. So every network output passes exactly through the waypoints, starts from the vehicle's state and is C3-continuous by construction. Derivatives are predicted in duration-normalised, standardised units ($x_d\, \tau^d$, with τ a geometric time scale from segment lengths). With fixed scales the jerk targets had a standard deviation of about 0.02, so jerk errors barely counted in the loss (Section 10).

Two inference modes are evaluated: **full neural** (durations and derivatives from the network, no solver on board) and **hybrid** (durations from the network, derivatives from one closed-form QP solve). Both pass through the same safety layer as the classical planner.

5.8 DAgger for receding-horizon re-planning

The network is trained on states that lie on teacher trajectories. In flight, however, it re-plans every 0.1 s from states on its *own* previous plan. Small duration biases compound, and each re-plan comes out slightly slower than the last. This is the classic covariate shift of imitation learning. NeuroSnap applies DAgger (Ross et al., 2011) in three rounds. Each round runs the current network in planner-only receding-horizon rollouts on 1,500 fresh courses (about 240,000 visited states). It labels 30,000 of those states with the teacher, adds them to the training set and fine-tunes for 15 epochs.

5.9 Safety layer, plan verifier and learned residual dynamics

Safety layer. If a trajectory exceeds the velocity or acceleration limit, its durations are stretched and, for the QP planners, the derivatives are re-solved. The start state is kept exactly.

Plan verifier. Before any re-plan, classical or learned, replaces the current trajectory, it must pass two cheap analytic checks. It must not reverse direction along its first segment (the overshoot-and-return loop), and it must not reach the next gate more than 1.4× later than the current plan. Otherwise the current plan is kept for another 0.1 s. Re-planning also stops within 0.7 m of a gate.

Residual dynamics. A 4→32→32→3 tanh MLP maps body-frame velocity and summed rotor speed to the acceleration that the thrust model does not explain (rotor drag and wind). It was trained on 215,333 samples from 35 logged flights and explains 86 % of the residual (0.234 → 0.034 m/s² RMS on held-out flights). It runs in NumPy at the 500 Hz control rate.

5.10 Live demo dashboard

The dashboard runs the complete loop and can be shown live in an OpenCV window or recorded to MP4 or GIF. It shows the top view (true gates, prior map, gate estimates with uncertainty, the current plan and the flown path), the onboard camera with detections, PnP ranges and confidence, and a HUD with time, speed, altitude, gates passed, planner and perception latency and a speed trace. Command-line options switch the planner (neural, hybrid, QP with optimised or heuristic times, straight line), the perception mode (camera, blind, oracle) and the residual model.

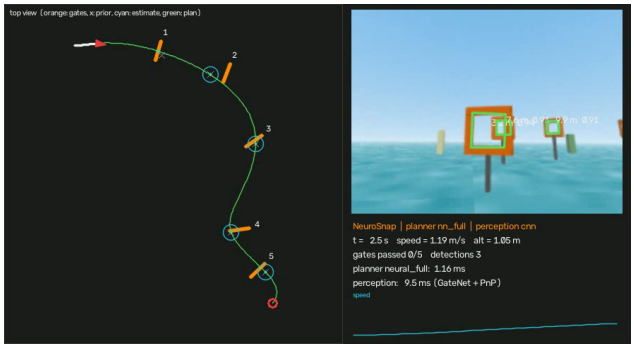


Figure 4a. Early in the mission: gate estimates (cyan) are being pulled from the prior (x) towards the true gates.

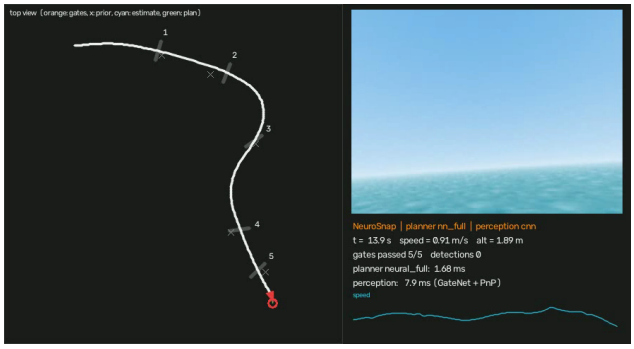


Figure 4b. Mission complete: all five gates passed, 13.9 s; the speed trace shows the smooth profile of the learned trajectory.

6. Experimental Setup and Metrics

Courses are generated at random with 3–7 gates, 2.5–5 m apart, with heading changes up to ±60°, heights from 0.8 to 2.0 m, gate yaw jitter of ±20° and 4–12 distractor objects. The finish point lies 2 m after the last gate. The benchmark uses 40 course seeds that were never used during development. Every method flies exactly the same courses, prior maps and wind. Eleven methods were compared:

- straight-line waypoint follower at 2.2 m/s and at 3.0 m/s (the baseline from the proposal);
- minimum-snap QP with heuristic durations, and with optimised durations (the reference paper);

- the neural planner in hybrid and full mode, and the full mode without DAgger;
- **NeuroSnap**: full neural planner + GateNet + residual model;
- NeuroSnap without the plan verifier, NeuroSnap flying the prior map only ("blind"), and NeuroSnap with the true gate positions ("oracle", an upper bound).

A mission succeeds if the drone passes every gate in order, without touching a frame, a stand or the ground, and stops within 0.35 m of the finish within 40 s. A crossing of a gate plane outside the frame counts as a missed gate. The metrics are those listed in Section 8.7 of the initial report:

- detection precision and recall, corner error and PnP position error against range;
- trajectory quality: snap cost and duration relative to the minimum-snap optimum;
- tracking performance: RMSE of the flown path against the reference;
- mission success rate, gates passed and lap time;
- compute time: planner time per call against the QP, and perception latency;
- ablations: Transformer against MLP, snap loss, DAgger, verifier, residual model, perception mode.

7. Results

7.1 Learned planner against the optimiser (1,000 held-out problems)

Planner	Snap / optimal (median)	Snap p90	Duration / optimal	ms per call (mean / p95)
Min-snap QP, heuristic durations	8.59×	66.2×	1.054	0.50 / 1.00
Min-snap QP, optimised durations (teacher)	1.00×	1.0×	1.000	2.58 / 4.19
Transformer + DAgger, full neural	**2.01×**	**9.1×**	**1.001**	**0.90 / 1.23**
Transformer + DAgger, hybrid	**1.00×**	**1.1×**	**1.001**	**0.92 / 1.17**
Transformer, behaviour cloning only	2.31×	11.2×	1.002	0.85 / 1.16
Transformer, + snap-gap loss 0.1	3.10×	19.4×	1.002	0.85 / 1.17
Transformer, + snap-gap loss 0.5	4.68×	38.3×	1.005	0.88 / 1.24
MLP (same decoder, snap loss 0.5)	17.03×	124.5×	1.023	0.57 / 0.90

The **hybrid mode matches the optimiser** (1.00× snap, durations within 0.1 %) at 2.8× lower mean and 3.6× lower p95 compute: the network replaces the iterative time allocation, and a single linear solve recovers the optimal derivatives. The **full-neural mode** needs no solver at all. Its durations are just as accurate, but its trajectories carry 2.0× the minimum snap, because snap is very sensitive to small jerk errors at the knots. Attention matters: the MLP with the same decoder is 8× worse. The snap-gap loss is a negative result. It lowers snap at the teacher's durations, but it makes the predicted durations less accurate, and since snap scales with T^{-7}, duration accuracy dominates end to end.

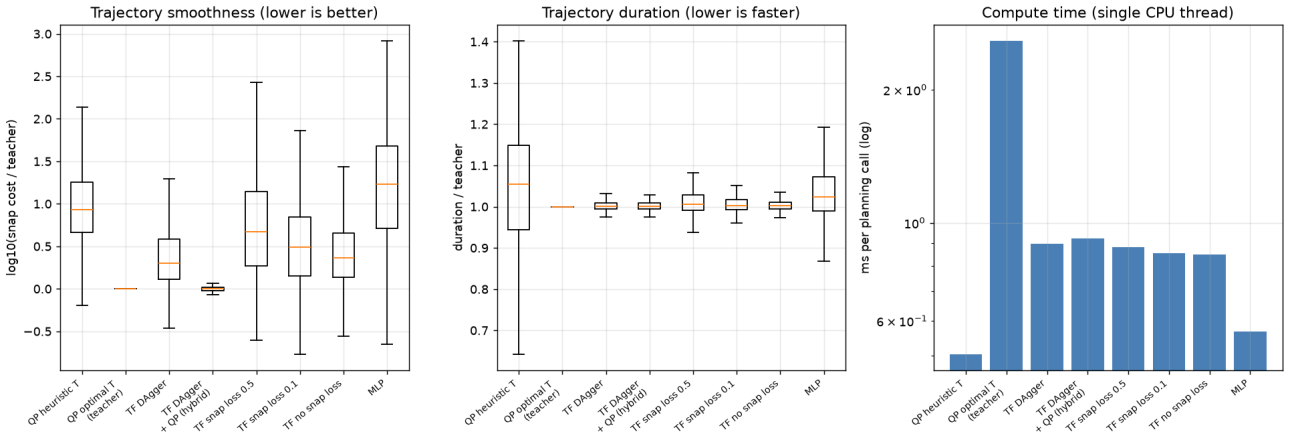


Figure 5. Snap cost and duration relative to the optimiser (box plots, outliers hidden) and compute time per planning call, single CPU thread.

7.2 DAgger and receding-horizon drift

Planner	Drift median	Drift p90	Blow-ups (> 3×) / 100
Teacher (min-snap, optimised durations)	1.25	1.60	0
Transformer, behaviour cloning	1.90	2.46	8
+ DAgger round 1 (23,991 states)	1.62	2.00	3
+ DAgger round 2 (25,022 states)	1.49	1.79	5
+ DAgger round 3 (25,123 states)	**1.44**	**1.78**	**2**

Drift is the time to finish when re-planning every 0.1 s, divided by the duration of the first plan (planner-only rollouts on 100 unseen courses). DAgger cuts the slow-down from 1.90× to 1.44× and the blow-ups from 8 to 2. It also improved the offline snap ratio from 2.31× to 2.01×. The teacher itself drifts by 1.25×, because its time-optimal tail problems are re-solved from a fixed start state.

7.3 Perception (1,000 held-out frames, 1,504 gates within 8 m)

Metric	Value
Detection precision / recall	0.89 / 0.89
Corner keypoint error (median)	1.19 px
Gate-centre error after PnP (median, all ranges)	9.0 cm
Latency: GateNet + decoding + PnP (CPU, mean / p95)	8.7 / 12.8 ms

Range to gate	Frames	Corner error	Position error (median)	Position error (p90)	KF σ used
0-2 m	331	2.92 px	4.8 cm	9.3 cm	5.2 cm
2-4 m	415	1.07 px	7.3 cm	21.4 cm	14.8 cm
4-6 m	320	1.01 px	17.6 cm	60.2 cm	34.0 cm
6-8 m	279	0.89 px	34.3 cm	99.2 cm	62.8 cm

Corner error in pixels is almost constant with range, so the 3-D error grows roughly with distance squared, as PnP theory predicts. This is why far fixes are down-weighted in the Kalman filter and fixes beyond 7 m are ignored. Close gates have a larger pixel error but still the best 3-D accuracy (4.8 cm), even when their corners fall outside the image.

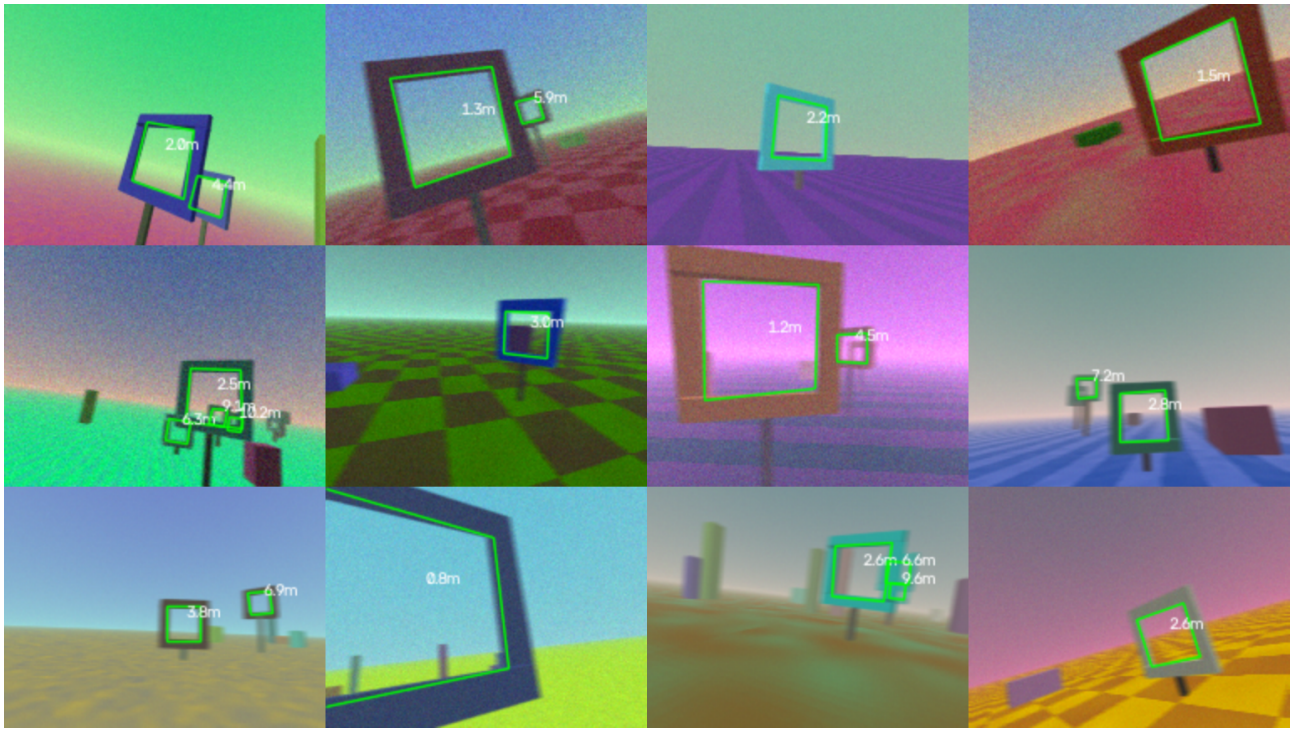


Figure 6. Held-out validation frames with GateNet detections (green) and PnP range, across randomised colours, textures, lighting, noise and blur.

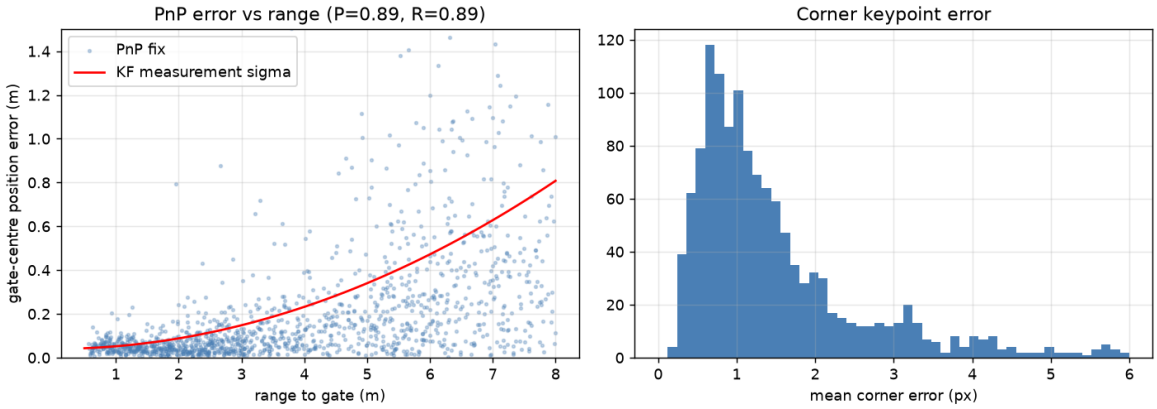


Figure 7. PnP gate-centre error against range with the Kalman-filter measurement model (left) and the corner-keypoint error distribution (right).

7.4 Closed-loop benchmark (440 missions)

Method	Success	Crash	Missed	Gates	Lap (s)	RMSE (cm)	Gate err (cm)
Straight line 2.2 m/s	100 %	0 %	0 %	100 %	9.4	14.5	2.8
Straight line 3.0 m/s	100 %	0 %	0 %	100 %	7.0	24.7	3.7
Min-snap, heuristic T	87.5 %	5 %	7.5 %	91.9 %	13.5	3.7	7.7
Min-snap, optimised T (paper)	100 %	0 %	0 %	100 %	13.6	4.0	4.0
Neural hybrid	92.5 %	5 %	2.5 %	96 %	15.0	3.7	6.8
Neural full	92.5 %	5 %	2.5 %	97.2 %	14.9	3.6	4.4
Neural full, no DAgger	85 %	10 %	2.5 %	91.3 %	16.8	3.2	5.7
NeuroSnap (+ residual)	**97.5 %**	**2.5 %**	**0 %**	**98.1 %**	**15.2**	**1.7**	**3.5**
NeuroSnap, no verifier	82.5 %	7.5 %	7.5 %	90.7 %	16.3	145.7*	4.1
NeuroSnap, blind	0 %	62.5 %	37.5 %	15.1 %	–	1.8	49.2
NeuroSnap, oracle gates	100 %	0 %	0 %	100 %	10.6	1.8	0.0

All methods except "blind" and "oracle" use GateNet perception. * One run without the verifier accepted a bad plan and diverged, which dominates the mean.

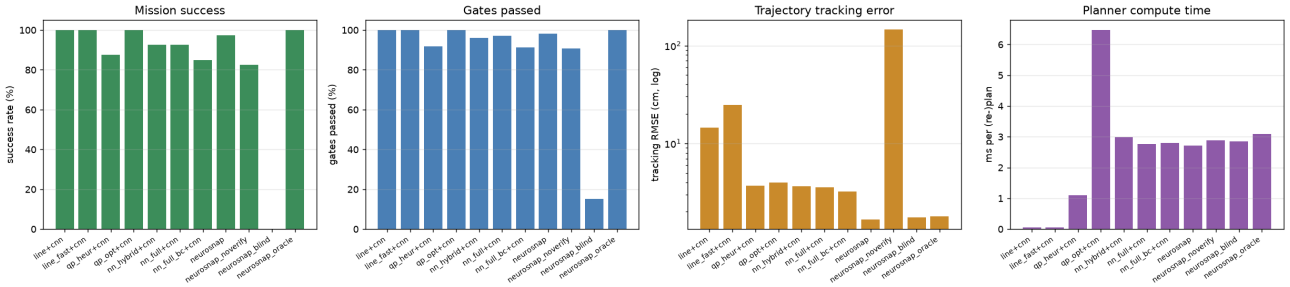


Figure 8. Success rate, gates passed, tracking RMSE (log scale) and planner compute time per method. Planner times here include seven missions running in parallel.

Perception is what makes the task solvable. Flying the prior map completes 0 % of courses. With GateNet, PnP and the Kalman map, the gate-position error falls from 49 cm to 3.5 cm and NeuroSnap completes 97.5 %. **The learned stack comes within 2.5 points of the classical planner**, with 2.4× lower tracking error (thanks to the residual model) and no optimiser on board. **DAgger and the verifier are both needed:** removing DAgger costs 7.5 points, and removing the verifier costs 15. NeuroSnap is still about 12 % slower per lap than the classical planner (15.2 s against 13.6 s). This is the remaining receding-horizon drift: without re-planning (oracle) the same network flies the course in 10.6 s.

> **The straight-line follower is a strong baseline in this setup.** The 0.8 m gates are generous for a 12 cm vehicle, and a straight line goes exactly through the gate centres, so it never fails and is the fastest. The price is 14.5–24.7 cm of tracking error, 4–15× more than the smooth planners: the vehicle cannot follow corners it was never planned to fly. This dynamic infeasibility is exactly what minimum-snap planning exists to remove, and with tighter gates or faster flight it would cause failures. The result is reported as measured.

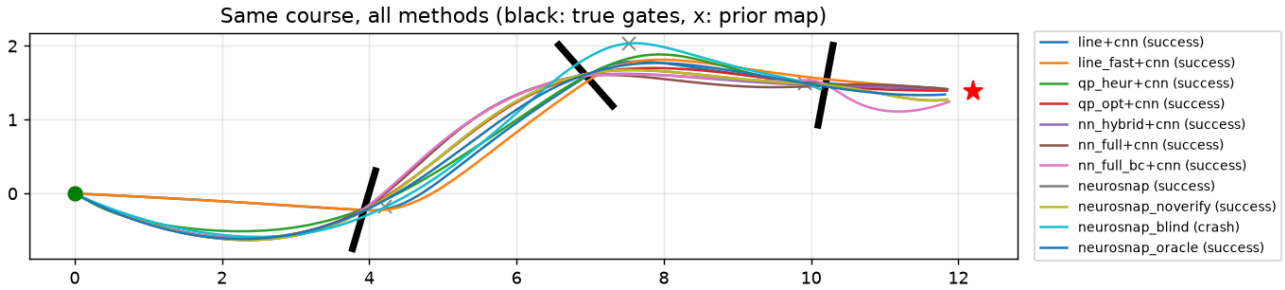


Figure 9. The paths flown by all eleven methods on the same course (black: true gates, x: prior map). Only the blind variant, which aims at the prior map, fails.

7.5 Real-time performance

Per camera frame, GateNet with decoding and PnP takes 8.7 ms on the CPU (p95 12.8 ms). The neural planner takes 0.90 ms per re-plan (hybrid 0.92 ms; classical optimiser 2.58 ms, p95 4.19 ms), and the residual model runs in NumPy at 500 Hz. The complete perception-to-plan chain therefore takes about 10 ms, well inside the 66 ms budget of a 15 Hz camera.

8. Stability and Control Analysis (Syllabus Module 5)

Step responses. The cascade was tested on the full nonlinear model with attitude-level steps (the position loop bypassed), an altitude step through the full cascade, and a horizontal gust at hover.

Axis / test	Result
Roll (lateral) and pitch (longitudinal) 10° step	rise time 0.08 s, overshoot 0.0 %, 5 % settling 0.19 s
Yaw (directional) 45° step	rise time 0.25 s, overshoot 0.0 %, settling 0.38 s
Altitude +0.5 m step	rise time 1.01 s, overshoot 2.3 %, settling 1.41 s
Gust: 0.05 N (19 % of weight) for 0.5 s	peak deviation 16.3 cm, back within 2 cm after 1.1 s
Roll rate loop (PI + motor lag + 1-sample delay)	phase margin 37°, gain margin 12.7 dB, crossover 45 rad/s
Roll attitude loop	phase margin 75°, gain margin 7.3 dB, crossover 15 rad/s
Stability boundary	attitude loop unstable only for K_{att} > 33 (design 14: 2.4× margin, consistent with the 7.3 dB gain margin)

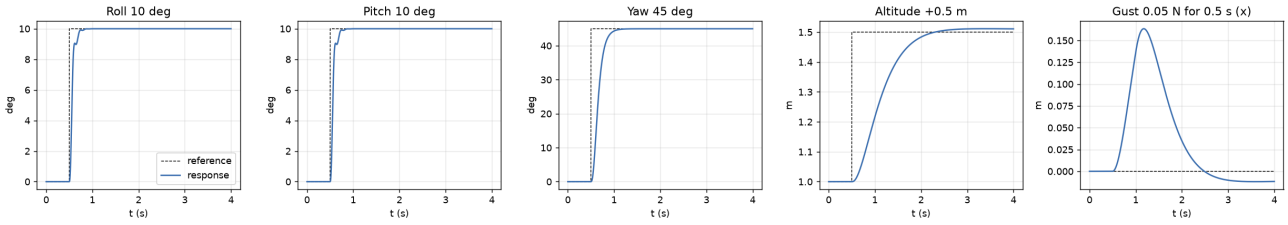


Figure 10. Closed-loop responses on the nonlinear model: roll, pitch, yaw, altitude and gust rejection.

Linear analysis. About hover the roll axis reduces to a double integrator, because the controller cancels the inertia. The rate loop is PI control through the motor lag (τ = 20 ms) and one 2 ms control period of delay (first-order Padé), and the attitude loop is P control around the closed rate loop. The bare airframe has no restoring moment, so all stability comes from feedback. The rate loop has the thinner margin (37°), which is the main limit on more aggressive attitude gains.

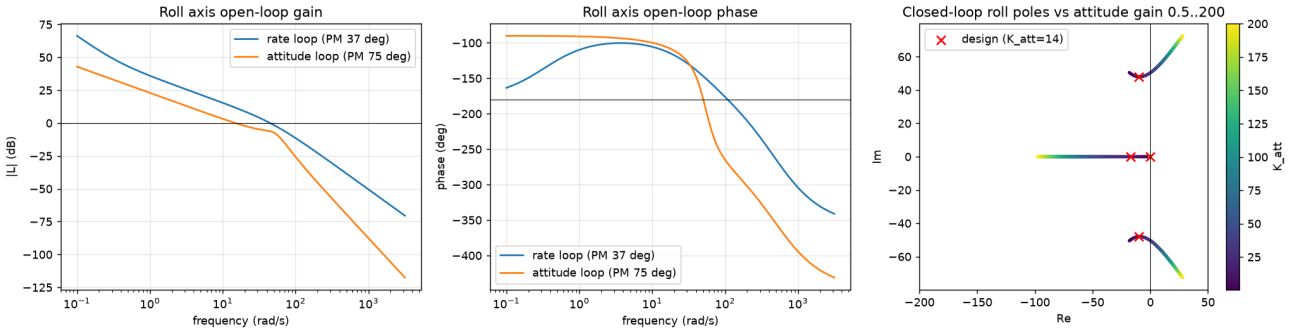


Figure 11. Roll-axis Bode plots of the rate and attitude loops (left, centre) and closed-loop poles as the attitude gain varies from 0.5 to 200 (right).

9. Mapping to the BCSE428L Syllabus

Module	Where it appears in NeuroSnap
1 – Introduction (types, airframe, batteries, motors, ESC, propellers)	Quad-X Crazyflie airframe, rotor thrust/torque model, motor lag, rotor drag (Sec. 5.1)
2 – Design fundamentals (flight controllers, RC, FPV, telemetry)	Cascaded flight controller and mixer, FPV camera, dashboard with flight HUD (Sec. 5.2, 5.3, 5.10)
3 – Drone basics (flight modes, roll/pitch/yaw, GPS autopilot, intelligent modes)	Autonomous gate-racing mission, attitude and yaw control, waypoint autopilot with re-planning (Sec. 5.2, 5.8, 5.9)
4 – Modelling and control	6-DOF Newton–Euler model, differential flatness and minimum-snap trajectories, learned residual dynamics (Sec. 5.1, 5.6, 5.9)
5 – Stability and control	Roll, pitch, yaw and altitude steps, gust rejection, Bode margins, pole locus (Sec. 8)
6 – Applications	Autonomous drone racing and gate inspection with an approximate map (Sec. 6)
7 – Expanding drone abilities (camera, FPV, sensors, speed)	Monocular camera with deep perception, faster planning on board, speed and acceleration limits (Sec. 5.3–5.7, 7.5)

10. Challenges Faced and How They Were Solved

- **Dataset generation hung.** Some mid-flight problems with perturbed start states produced degenerate teacher solutions, whose limit stretching exploded the trajectory duration. Sampling is now capped, and problems that need more than 3× stretching are rejected.
- **Jerk errors were invisible to the loss.** With fixed output scales, the normalised jerk targets had a standard deviation of 0.02, and the first planner reached 9× the optimal snap. Predicting duration-normalised, standardised derivatives fixed the scaling. A first attempt that normalised with the *predicted* durations coupled the two outputs and made timing worse; a geometric time scale from the inputs solved it.
- **The snap-gap loss did not help.** A fair comparison at 60 epochs showed that plain imitation gives the best end-to-end snap, because duration accuracy dominates. The loss was kept as an ablation and reported as a negative result.
- **The drone kept slowing down in flight.** Receding-horizon re-planning compounded small duration biases (1.90× drift). DAgger brought this down to 1.44×.

- **Overshoot-and-return loops.** Just before a gate, the network sometimes predicted a long first segment while the start velocity was fixed. The only polynomial that fits overshoots and comes back, and the drone hit the gate it had just passed. The analytic plan verifier rejects such plans.
- **The first verifier was unfair to the baselines.** A rule that rejected any stretched plan blocked 34 % of the classical planner's re-plans and dropped its success to 77.5 %. The rule was removed, since the reversal check already catches the loop, and the whole benchmark was re-run.
- **Integrator wind-up.** On large steps the position integrator wound up and the drone settled slowly. Conditional integration and a lower integral gain fixed it.
- **Non-deterministic benchmark.** Renderer noise used the global NumPy RNG, so re-running a mission gave different outcomes. Every renderer now owns a seeded generator, and missions are reproducible.
- **Unfair perception scoring.** Correct detections of gates beyond 8 m were counted as false positives. They are now ignored, as in the COCO "ignore" convention, which raised measured precision from 0.74 to 0.89.

11. Limitations and Future Work

- **Simulation only.** The renderer is far simpler than a real camera (no lens distortion, rolling shutter or real textures). Real flight would need GateNet fine-tuned on real images and a real state estimator instead of the simulator's state plus noise.
- **Generous gates.** The 0.8 m opening makes the task forgiving, which is why the straight-line baseline succeeds. Smaller gates and higher limits would separate the planners more clearly.
- **Remaining drift.** After DAgger, re-planning is still slower than the teacher (1.44× vs 1.25×). More DAgger rounds, or a loss that rewards consistency between consecutive plans, are the next steps.
- **The verifier is a heuristic.** It catches overshoot loops and sudden slow-downs, but not every bad plan (one 2.5 % crash remains).
- **No obstacle avoidance and no yaw planning.** Distractor objects are only visual, and the heading simply points at the next gate. The reference paper also plans yaw. Moving to gym-pybullet-drones or PX4/Gazebo and real hardware are further future work.

12. How to Run

```
python3.12 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python scripts/demo.py --seed 3 --live          # live dashboard (trained weights included)
python scripts/demo.py --planner qp_opt         # same course with the classical planner
python scripts/evaluate_planner.py --n 1000     # offline planner comparison
python scripts/evaluate_perception.py --n 1000  # GateNet + PnP evaluation
python scripts/benchmark.py --seeds 40          # closed-loop benchmark (11 methods)
python scripts/control_analysis.py              # Module 5 analysis
python -m pytest -q                             # 14 unit / integration tests
sh scripts/reproduce.sh                         # everything from scratch
```

13. Conclusion

The project set out to keep the reference paper's core idea, smooth and dynamically feasible minimum-snap trajectories through waypoints as the interface between perception and control, while replacing its run-time mathematics with a deep-learning pipeline. NeuroSnap implements this end to end:

- a self-labelled, domain-randomised perception pipeline (GateNet + PnP + Kalman map);
- a Transformer that predicts minimum-snap trajectories through a structure-preserving decoder, trained by imitation of the paper's optimiser and corrected with DAgger;
- an analytic verifier and safety layer around the learned planner;
- a cascaded flight controller with a learned drag model, analysed for stability.

With 97.5 % mission success (0 % blind, 100 % for the classical planner), 1.7 cm tracking error, near-optimal time allocation at 2.8× lower planning cost and about 10 ms per camera frame, the pipeline meets the objectives of the initial report. The evaluation also shows honestly where the learned planner still falls short of the optimiser: lap time and full-neural smoothness.

References

1 D. Mellinger and V. Kumar, "Minimum Snap Trajectory Generation and Control for Quadrotors," in *Proc. IEEE ICRA*, Shanghai, 2011, pp. 2520–2525. doi:10.1109/ICRA.2011.5980409

2 C. Richter, A. Bry, and N. Roy, "Polynomial Trajectory Planning for Aggressive Quadrotor Flight in Dense Indoor Environments," in *Proc. ISRR*, 2013.

3 S. Tankasala and M. Pryor, "Accelerating Trajectory Generation for Quadrotors Using Transformers," *L4DC*, 2023. arXiv:2303.15606

4 A. Vaswani et al., "Attention Is All You Need," *NeurIPS*, 2017. arXiv:1706.03762

5 S. Ross, G. Gordon, and D. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," *AISTATS*, 2011. arXiv:1011.0686

6 X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," 2019. arXiv:1904.07850

7 T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," 2017. arXiv:1708.02002

8 T. Collins and A. Bartoli, "Infinitesimal Plane-Based Pose Estimation," *International Journal of Computer Vision*, vol. 109, 2014.

9 J. Tobin et al., "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," *IROS*, 2017. arXiv:1703.06907

10 E. Kaufmann et al., "Beauty and the Beast: Optimal Methods Meet Learning for Drone Racing," *ICRA*, 2019. arXiv:1810.06224

11 A. Loquercio et al., "Learning High-Speed Flight in the Wild," *Science Robotics*, vol. 6, no. 59, 2021. doi:10.1126/scirobotics.abg5810

12 T. Lee, M. Leok, and N. H. McClamroch, "Geometric Tracking Control of a Quadrotor UAV on SE(3)," in *Proc. IEEE CDC*, 2010.

13 J. Panerati et al., "Learning to Fly – a Gym Environment with PyBullet Physics for Reinforcement Learning of Multi-agent Quadcopter Control," *IROS*, 2021.

14 R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, vol. 82, 1960.

15 PyTorch, pytorch.org · OpenCV, opencv.org · SciPy, scipy.org